Feature Model Configuration as SAT: Group Project Report - Part I

Ana Carvalhal(up202305274), André Souza(202109775), João Taveira (up202306063), Rodrigo Rocha(202305891)

I. Introduction

For the Advanced Software Construction Techniques (TACS) class project, students were assigned a random task from 24 available assignments. Our group was assigned the B theme: Search with Logical Constraints, specifically the SAT problem. This report covers the first part of the project, which includes a model domain analysis, Problem DSL, Solution DSL, OCL constraints, and an object model example. This report will document our analysis of feature models and their mapping to CNF formulas in order to design the two languages of our chain. This gives us the basis to create meta-models for feature models and their propositional meaning.

II. Process

A. Technologies

We made our deliverables using the recommended technologies to properly reach a reasonable result. The selected technologies were: B-UML for metamodels; BESSER's Object Models for object models; BESSER's OCL (BOCL) for restrictions; Python for M2M transformations; Git for version control. We chose this stack mainly because of the reference implementation: the teachers provide a reference implementation of the full chain for the Annex B example, built in BESSER (Part 0). Using the same stack lets us follow that example directly.

B. Domain Analysis and Concept Extraction

We've based our analysis on the work of Batory to understand how a feature model is transformed into a propositional formula, and also on the definition of CNF given for class B in the assignment.

As a development example, we followed the Party Planning problem from the course slides. It is a small, simple and easy-to-understand example, which helped us understand features, groups, constraints, and solutions.

The first step was identifying the concepts needed for our feature model: features, hierarchy, groups of features and constraints between branches of the tree. The second step was identifying the concepts of a CNF formula: variables, literals, and clauses. For each concept, we defined its attributes and their connection to other concepts. Some highlights of our analysis were:

- Every feature model needs to have exactly one root element, which will always be selected.
- The mandatory or optionality of a feature is treated as a property of the feature itself (`isMandatory`) and not as a separate concept. The only difference between a mandatory and an optional child is the clause they make with their parent: an optional child generates "if there is a child there is a parent" while a mandatory also generates "if there is a parent there is a child"
- Or and Xor groups store exactly the same information: a parent and its members. They only differ in the selection rule: Or requires choosing *at least one* member when the parent is selected, while Xor requires choosing *exactly one*. To make a group meaningful, an Or group must have at least one member and an Xor group at least two, which is why they have a common structure and share an abstract superclass.
- A positive literal A and a negative $\neg A$ only differ in having or not the "$\neg$", so we only needed a boolean (`isNegated`) on the literal instead of two classes.

Building upon these core observations, we structured the domain concepts across both languages as follows:

C. Problem Domain Concepts

The structural core of the Problem DSL is the `Feature`, characterized by a unique `id`, a human-readable `name`, and the `isMandatory` flag. The feature `id` serves as the canonical identifier for its corresponding boolean variable in the CNF formula. Group variability is handled by the abstract superclass `GroupFeature`, which connects a parent feature to its members. Its concrete subclasses define the selection rules: an `Or` group requires at least one member to be active when the parent is selected (expressed via $(\neg P \vee F_1 \vee \cdots \vee F_n)$ plus child implies parent clauses $(\neg F_i \vee P)$), while an `Xor` group enforces mutual exclusion by adding pairwise negative clauses $(\neg F_i \vee \neg F_i)$ for every combination of members.

Non-hierarchical dependencies between distinct branches are captured by `CrossTreeConstraints`, which unifies relationships between a `source` and a `target` feature: a `RequiresConstraint` ($A \implies B$) yields ($\neg A \vee B$), and an `ExcludesCOnstraint` ($A \implies \neg B$) translates to ($\neg A \vee \neg B$). Finally, user-driven configurations are decoupled from the tree using `SelectionRequest` and `FeatureChoice`. Each choice marks a feature as `REQUIRED` or `EXCLUDED`, translating into unit clauses (X)

and $(\neg X)$, thereby enabling mutiple configuration scenarios over teh same immutable feature model.

1) Solution Domain Concepts: The Solution DSL formalities the propositional CNF representation required by Grover´s quantum search algorithm, A `CnfFormula` aggregates a set of disjunctive `Clause` instances, each containing one or more `Literal` elements referencing a target `Variable`. The truth polarity of each literal is governed by the `isNegated` attribute.

Each `Variable` is assigned a unique, contiguous 0-based register `index`, denoting its exact qubit position within the quantum search register. Lastly, `SolverConfig` encapsulates the quantum execution parameters, maintaining the sampling count (`shots`), the expected solution count (`knownSolutions`), and the optimal Grover iteration count (`rounds`), evaluated analytically as $\lfloor \frac{\pi}{4}\sqrt{N/M} \rfloor$.

III. Concept Tables and Meta-models

As a result of our analysis, we developed two concept tables, Table I for the problem DSL and Table II for the solution DSL, both of them are available in the project repository.

1) Table I - Problem DSL:

- Stores the list of concepts of the Problem DSL with their classes, attributes and connection with other concepts while reference the Solution DSL it is mapped to.
- Defines the structural relationships between features, hierarchy, feature groups, and cross-tree constraints.
- Specifies how feature selection requests and constraints are translated into logical rules for generating the corresponding CNF formula.

2) Table II - Solution DSL:

- Stores the list of concepts used to represent the solution, including their attributes and relationships with other concepts.
- Defines the structure of the CNF formula through variables, clauses, and literals, representing the problem as a logical expression.
- Specifies the configuration parameters of the solver, such as the number of rounds, shots, and known solutions, required for solving the formula.

IV. OCL Constraints

The Object Constraint Language (OCL) was used to define additional restrictions on the Problem DSL and Solution DSL meta-models, ensuring that they comply with the structural and semantic rules of the feature modelling domain.

In the Problem DSL, the constraints primarily ensure the consistency of the feature model. Each `FeatureModel` must have a root feature and contain at least one feature. The root must belong to the model, cannot have a parent, and cannot be mandatory. Furthermore, `Feature` identifiers must be unique within the same model to prevent ambiguity when identifying and selecting features, and each identifier must be non-empty.

The `Feature` constraints enforce the validity of the feature hierarchy. A feature without a parent must be the root of its model, while all other features must have a parent belonging to the same model. Additionally, a feature cannot be its own parent, and cycles in the hierarchy are forbidden. These restrictions ensure that the feature model forms a valid hierarchy.

The `GroupFeature`, `Or`, and `Xor` constraints ensure consistency between feature groups and their members. Every member must be a child of the group's parent feature. An `Or` group must contain at least one member, while an `Xor` group must contain at least two. Moreover, members of these groups cannot be mandatory, since their selection is governed by the group constraints.

The `CrossTreeConstraint` constraints prevent a feature from being constrained against itself and ensure that both the source and target features belong to the same feature model that owns the constraint. This avoids invalid references between unrelated models.

Finally, the `SelectionRequest` and `FeatureChoice` constraints ensure that every selected feature belongs to the model associated with the corresponding request. A selection request cannot contain multiple choices for the same feature, preventing redundant or conflicting decisions regarding the same feature.

In the Solution DSL, the constraints ensure that the CNF formula can be encoded in the quantum register. A `CnfFormula` must contain at least one variable and clause and exactly one `SolverConfig`. Variable names and indices and clause identifiers are required to be unique within the formula, and the indices must be contiguous so that they map to valid qubit positions. The number of known solutions cannot exceed 2^n, where n is the number of variables. A `Variable` must have a non-empty name and a non-negative index. A `Clause` must have a non-empty identifier and at least one literal, cannot contain two literals on the same variable, and all its literals must reference variables of the same formula. Finally, the `SolverConfig` requires `rounds` and `shots` to be greater than zero and `knownSolutions` to be non-negative.

In summary, these constraints improve the integrity of the meta-models by preventing invalid configurations, inconsistent relationships, and ambiguous feature selections before the transformation into the CNF representation takes place.

V. Conclusion

In this report, we covered essential highlights of the first part of the project of TACS. We have reasonably and satisfyingly covered important details and we are looking forward to continue through the part 2.

VI. Repository

The source files for the meta-models and tables generated for this project are available in the TACS-GROUP06 repository.

References

[1] D. Batory, "Feature models, grammars, and propositional formulas," in *Proc. 9th Int. Software Product Line Conf. (SPLC)*, LNCS 3714, Springer, 2005, pp. 7–20.